



Итоговое практическое задание

Bash-скрипт «Системный помощник Ubuntu»

Практическое задание

Необходимо разработать Bash-скрипт `system_helper.sh`, который собирает основную информацию о системе Ubuntu и выполняет несколько простых административных операций.

Скрипт должен работать как небольшая консольная программа. Пользователь запускает его с аргументом, определяющим нужное действие.

Примеры запуска:

```
./system_helper.sh info
./system_helper.sh users
./system_helper.sh processes
./system_helper.sh network
./system_helper.sh service ssh
./system_helper.sh find /var/log "*.log"
./system_helper.sh backup /home/student/documents
./system_helper.sh report
./system_helper.sh help
```

Скрипт не должен выполнять опасные действия: удалять пользователей, завершать процессы, изменять системные файлы или останавливать службы.

Цель

Цель задания — закрепить навыки работы с Linux и Bash:

- создание и запуск Bash-скриптов;
- использование переменных;

- обработка аргументов командной строки;
- применение условий `if`, `elif`, `else`;
- использование конструкции `case`;
- применение циклов `for` и `while`;
- работа с файлами и директориями;
- проверка существования файлов, директорий и команд;
- обработка кодов завершения;
- перенаправление стандартного вывода и ошибок;
- использование конвейеров команд;
- работа с процессами;
- просмотр состояния служб;
- получение сетевой информации;
- поиск файлов;
- создание резервных копий;
- запись результатов в лог;
- базовая обработка ошибок.

В результате должен получиться один понятный и безопасный Bash-скрипт, который можно использовать для первичной диагностики Ubuntu.

Задание

Шаг 1. Подготовьте рабочую директорию

Создайте директорию проекта:

```
mkdir -p ~/linux-final-project  
cd ~/linux-final-project
```

Создайте основной файл:

```
nano system_helper.sh
```

В первой строке укажите интерпретатор Bash:

```
#!/bin/bash
```

Сделайте файл исполняемым:

```
chmod +x system_helper.sh
```

Структура проекта должна выглядеть примерно так:

```
linux-final-project/  
├─ system_helper.sh  
├─ backups/  
├─ logs/  
└─ reports/
```

Директории `backups` , `logs` и `reports` скрипт должен создавать автоматически, если они отсутствуют.

Шаг 2. Добавьте основные переменные

В начале скрипта создайте переменные:

```
SCRIPT_DIR=  
LOG_DIR=  
BACKUP_DIR=  
REPORT_DIR=  
LOG_FILE=
```

Переменная `SCRIPT_DIR` должна содержать абсолютный путь к директории, в которой находится скрипт.

Пример:

```
SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
```

Остальные пути должны строиться относительно `SCRIPT_DIR`.

Например:

```
LOG_DIR="$SCRIPT_DIR/logs"
BACKUP_DIR="$SCRIPT_DIR/backups"
REPORT_DIR="$SCRIPT_DIR/reports"
```

Создайте необходимые директории:

```
mkdir -p "$LOG_DIR" "$BACKUP_DIR" "$REPORT_DIR"
```

Шаг 3. Реализуйте функцию журналирования

Создайте функцию:

```
log_message()
```

Она должна принимать текст и записывать его в лог-файл вместе с датой и временем.

Пример записи:

```
2026-08-04 14:30:15 | Запущена команда info
```

Пример функции:

```
log_message() {
    local message="$1"
    echo "$(date '+%Y-%m-%d %H:%M:%S') | $message" >> "$LOG_FILE"
}
```

Лог-файл может называться:

```
logs/system_helper.log
```

В лог необходимо записывать:

- запуск скрипта;
- выбранную команду;
- успешное выполнение операции;
- ошибки;
- создание резервной копии;
- создание отчёта;
- проверку службы;
- поиск файлов.

Шаг 4. Реализуйте справку

Создайте функцию:

```
show_help()
```

Она должна выводить список доступных команд и примеры их использования.

Пример:

Использование:

```
./system_helper.sh info
./system_helper.sh users
./system_helper.sh processes
./system_helper.sh network
./system_helper.sh service <имя_службы>
./system_helper.sh find <директория> <шаблон>
./system_helper.sh backup <директория>
./system_helper.sh report
./system_helper.sh help
```

Для каждой команды добавьте короткое описание.

Шаг 5. Команда `info`

Команда:

```
./system_helper.sh info
```

должна выводить общую информацию о системе:

- имя текущего пользователя;
- имя компьютера;
- текущую дату и время;
- версию ядра Linux;
- название и версию операционной системы;
- архитектуру системы;
- время работы системы;
- текущую рабочую директорию;
- используемый shell;
- свободное место на дисках;
- информацию об оперативной памяти.

Можно использовать команды:

```
whoami  
hostname  
date  
uname  
uptime  
pwd  
df  
free  
cat /etc/os-release
```

Желательно выводить только основные строки, а не всё содержимое команд.

Пример результата:

```
=== Общая информация о системе ===
Пользователь: student
Компьютер: ubuntu-server
Дата: 04.08.2026 14:35
Операционная система: Ubuntu 24.04 LTS
Ядро: 6.8.0-31-generic
Архитектура: x86_64
Shell: /bin/bash
Время работы: 2 hours, 15 minutes
```

Шаг 6. Команда `users`

Команда:

```
./system_helper.sh users
```

должна выводить информацию о пользователях:

- текущего пользователя;
- его UID;
- его основную группу;
- все группы, в которые входит пользователь;
- домашнюю директорию;
- список пользователей, которые сейчас вошли в систему;
- количество обычных пользователей системы.

Можно использовать:

```
id
groups
who
getent passwd
```

Обычными пользователями можно считать учётные записи с UID не меньше `1000`.

Пример поиска:

```
getent passwd | cut -d: -f1,3,6
```

Для подсчёта пользователей разрешается применить `awk` :

```
getent passwd | awk -F: '$3 >= 1000 {print $1}'
```

Шаг 7. Команда `processes`

Команда:

```
./system_helper.sh processes
```

должна показывать:

- общее количество процессов;
- пять процессов с наибольшим потреблением процессора;
- пять процессов с наибольшим потреблением памяти;
- процессы текущего пользователя;
- наличие фоновых заданий текущего shell.

Можно использовать:

```
ps  
sort  
head  
wc  
jobs
```

Пример получения процессов с высокой нагрузкой на процессор:

```
ps aux --sort=-%cpu | head -n 6
```

Пример получения процессов с высоким потреблением памяти:


```
ps aux --sort=-%mem | head -n 6
```

Скрипт не должен автоматически завершать процессы.

Шаг 8. Команда `network`

Команда:

```
./system_helper.sh network
```

должна выводить:

- имя компьютера;
- список сетевых интерфейсов;
- IPv4-адреса;
- маршрут по умолчанию;
- используемые DNS-серверы;
- открытые TCP- и UDP-порты;
- результат проверки доступа к интернету.

Можно использовать:

```
hostname  
ip addr  
ip route  
ss  
ping
```

DNS-серверы можно получить из:

```
/etc/resolv.conf
```

Пример проверки сети:

```
ping -c 2 8.8.8.8
```

или:

```
ping -c 2 1.1.1.1
```

Скрипт должен проверить код завершения команды `ping`.

Пример логики:

```
if ping -c 2 8.8.8.8 > /dev/null 2>&1; then
    echo "Доступ к сети есть"
else
    echo "Не удалось выполнить проверку сети"
fi
```

Ошибка сети не должна аварийно завершать весь скрипт.

Шаг 9. Команда `service`

Команда:

```
./system_helper.sh service ssh
```

должна проверять состояние указанной службы.

Имя службы передаётся вторым аргументом:

```
$2
```

Скрипт должен:

1. Проверить, передано ли имя службы.
2. Проверить, существует ли такая служба.
3. Вывести её состояние.
4. Сообщить, запущена ли служба.
5. Сообщить, добавлена ли служба в автозагрузку.
6. Показать несколько последних строк журнала службы.

Можно использовать:

```
systemctl status
systemctl is-active
systemctl is-enabled
journalctl
```

Пример:

```
systemctl is-active --quiet "$service_name"
```

Проверка результата:

```
if systemctl is-active --quiet "$service_name"; then
    echo "Служба запущена"
else
    echo "Служба не запущена"
fi
```

Для просмотра журнала:

```
journalctl -u "$service_name" -n 10 --no-pager
```

Скрипт не должен запускать, останавливать или перезапускать службу.

Шаг 10. Команда `find`

Команда должна использоваться следующим образом:

```
./system_helper.sh find /var/log "*.log"
```

Первый аргумент после команды — директория поиска.

Второй аргумент — шаблон имени файла.

Скрипт должен:

1. Проверить наличие обоих аргументов.
2. Проверить существование директории.
3. Найти подходящие файлы.
4. Подсчитать количество найденных файлов.
5. Вывести найденные пути.
6. Записать информацию о поиске в лог.

Пример:

```
find "$search_dir" -type f -name "$pattern" 2>/dev/null
```

Результат поиска желательно сначала сохранить в переменную или временный файл.

Например:

```
results=$(find "$search_dir" -type f -name "$pattern" 2>/dev/null)
```

Количество результатов:

```
echo "$results" | grep -c .
```

Необходимо корректно обработать случай, когда ничего не найдено.

Шаг 11. Команда `backup`

Команда:

```
./system_helper.sh backup /home/student/documents
```

должна создавать архив указанной директории.

Скрипт должен:

1. Проверить, передан ли путь.
2. Проверить, существует ли директория.
3. Получить имя директории.

4. Сформировать имя архива с датой и временем.
5. Создать архив в директории `backups` .
6. Проверить код завершения команды `tar` .
7. Вывести размер созданного архива.
8. Записать результат в лог.

Пример имени архива:

```
documents_2026-08-04_14-50-20.tar.gz
```

Пример формирования даты:

```
timestamp=$(date '+%Y-%m-%d_%H-%M-%S')
```

Получение имени директории:

```
source_name=$(basename "$source_dir")
```

Создание архива:

```
tar -czf "$archive_path" -C "$(dirname "$source_dir")" "$source_name"
```

После выполнения необходимо проверить `$?` либо использовать команду внутри `if` .

Пример:

```
if tar -czf "$archive_path" -C "$(dirname "$source_dir")" "$source_name"; then
    echo "Резервная копия создана"
else
    echo "Ошибка создания резервной копии" >&2
    exit 1
fi
```

Скрипт не должен удалять исходные файлы.

Шаг 12. Команда `report`

Команда:

```
./system_helper.sh report
```

должна создавать текстовый отчёт о состоянии системы.

Отчёт необходимо сохранить в директории:

```
reports/
```

Пример имени:

```
system_report_2026-08-04_15-00-00.txt
```

В отчёт должны попасть:

- дата и время создания;
- имя пользователя;
- имя компьютера;
- версия Ubuntu;
- версия ядра;
- время работы системы;
- использование диска;
- использование памяти;
- IP-адреса;
- маршрут по умолчанию;
- открытые порты;
- количество процессов;
- пять процессов с наибольшим потреблением CPU;
- пять процессов с наибольшим потреблением памяти;
- последние пять системных сообщений из `journalctl`.

Для записи нескольких команд в один файл используйте группировку:

```
{  
    echo "Системный отчёт"  
    echo "Дата: $(date)"  
    echo  
    echo "=== Диск ==="  
    df -h  
    echo  
    echo "=== Память ==="  
    free -h  
} > "$report_file" 2>&1
```

После создания отчёта скрипт должен вывести:

- путь к файлу;
- размер файла;
- сообщение об успешном создании.

Шаг 13. Обработайте команды через `case`

Основная часть скрипта должна обрабатывать первый аргумент:

```
command="$1"
```

Используйте конструкцию:

```
case "$command" in
    info)
        show_system_info
        ;;
    users)
        show_users
        ;;
    processes)
        show_processes
        ;;
    network)
        show_network
        ;;
    service)
        check_service "$2"
        ;;
    find)
        find_files "$2" "$3"
        ;;
    backup)
        create_backup "$2"
        ;;
    report)
        create_report
        ;;
    help|--help|-h)
        show_help
        ;;
    "")
        echo "Ошибка: команда не указана."
        show_help
        exit 1
        ;;
    *)
        echo "Ошибка: неизвестная команда '$command'."
        show_help
        exit 1
        ;;
esac
```


Каждая крупная операция должна быть оформлена как отдельная функция.

Рекомендуемые функции:

```
show_help
log_message
show_system_info
show_users
show_processes
show_network
check_service
find_files
create_backup
create_report
```

Шаг 14. Добавьте проверку команд

Перед использованием некоторых программ скрипт должен проверять, установлены ли они.

Создайте функцию:

```
check_command()
```

Пример:

```
check_command() {
    local command_name="$1"

    if ! command -v "$command_name" > /dev/null 2>&1; then
        echo "Ошибка: команда '$command_name' не установлена." >&2
        log_message "Команда $command_name не найдена"
        return 1
    fi
}
```

Функцию можно использовать для проверки:

```
ip  
ss  
tar  
systemctl  
journalctl
```

Не нужно автоматически устанавливать отсутствующие пакеты.

Разрешается вывести подсказку:

```
Установите пакет командой:  
sudo apt install iproute2
```

Шаг 15. Реализуйте обработку ошибок

Скрипт должен корректно обрабатывать следующие ситуации:

- команда не указана;
- передана неизвестная команда;
- отсутствует необходимый аргумент;
- директория не существует;
- файл или директория недоступны для чтения;
- команда отсутствует в системе;
- служба не существует;
- архив не удалось создать;
- отчёт не удалось записать;
- поиск завершился без результатов;
- для чтения системного журнала недостаточно прав.

Сообщения об ошибках выводите в поток ошибок:

```
echo "Ошибка: директория не существует." >&2
```

При критической ошибке используйте ненулевой код завершения:

```
exit 1
```

При успешной работе:

```
exit 0
```

Шаг 16. Используйте цикл

В скрипте должен использоваться хотя бы один цикл.

Например, цикл можно применить для проверки списка команд:

```
required_commands=("uname" "df" "free" "ps" "tar")

for cmd in "${required_commands[@]}; do
    if command -v "$cmd" > /dev/null 2>&1; then
        echo "[OK] $cmd"
    else
        echo "[ОШИБКА] $cmd не найден"
    fi
done
```

Также цикл можно использовать для красивого вывода сетевых интерфейсов или файлов, найденных командой `find`.

Дополнительная часть

Дополнительные пункты не являются обязательными, но улучшают итоговую работу.

1. Интерактивное меню

При запуске без аргументов можно предложить меню:

1. Информация о системе
2. Пользователи
3. Процессы
4. Сеть
5. Проверка службы
6. Поиск файлов
7. Резервное копирование
8. Создание отчёта
0. Выход

Для меню можно использовать цикл:

```
while true; do
    read -p "Выберите действие: " choice

    case "$choice" in
        1) show_system_info ;;
        2) show_users ;;
        3) show_processes ;;
        4) show_network ;;
        0) break ;;
        *) echo "Неверный выбор" ;;
    esac
done
```

2. Цветной вывод

Можно добавить цветовые переменные:

```
GREEN='\033[0;32m'
RED='\033[0;31m'
YELLOW='\033[0;33m'
NC='\033[0m'
```

Пример:

```
echo -e "${GREEN}Операция выполнена успешно${NC}"
```

Скрипт должен оставаться понятным и без цветного вывода.

3. Ротация резервных копий

Можно добавить удаление старых архивов, если их стало больше пяти.

Удалять разрешается только архивы, созданные самим скриптом внутри директории `backups`.

4. Планировщик cron

Добавьте в файл `README.txt` пример автоматического создания отчёта каждый день в 18:00:

```
0 18 * * * /home/student/linux-final-project/system_helper.sh report
```

Сам скрипт не должен автоматически изменять `crontab`.

5. Конфигурационный файл

Можно создать файл:

```
system_helper.conf
```

Пример:

```
PING_HOST="8.8.8.8"  
MAX_BACKUPS=5  
REPORT_PROCESS_COUNT=5
```

Скрипт может загружать его:

```
source "$SCRIPT_DIR/system_helper.conf"
```

Перед загрузкой необходимо проверить существование файла.

Подсказки по ключевым частям

Получение аргументов

```
command="$1"  
second_argument="$2"  
third_argument="$3"
```

Количество аргументов:

```
$#
```

Проверка аргумента:

```
if [ -z "$1" ]; then  
    echo "Аргумент не указан." >&2  
    exit 1  
fi
```

Проверка файла или директории

Проверка директории:

```
if [ -d "$path" ]; then  
    echo "Директория существует"  
fi
```

Проверка обычного файла:

```
if [ -f "$path" ]; then
    echo "Файл существует"
fi
```

Проверка доступности для чтения:

```
if [ -r "$path" ]; then
    echo "Объект доступен для чтения"
fi
```

Проверка возможности записи:

```
if [ -w "$path" ]; then
    echo "Доступна запись"
fi
```

Работа с кодом завершения

Каждая команда возвращает код завершения.

```
command
exit_code=$?
```

Код `0` обычно означает успешное выполнение.

```
if [ "$exit_code" -eq 0 ]; then
    echo "Команда выполнена успешно"
else
    echo "Команда завершилась с ошибкой"
fi
```

Удобнее проверять команду напрямую:

```
if mkdir -p "$BACKUP_DIR"; then
    echo "Директория готова"
else
    echo "Не удалось создать директорию" >&2
    exit 1
fi
```

Перенаправление вывода

Запись с заменой содержимого:

```
command > file.txt
```

Добавление в конец файла:

```
command >> file.txt
```

Запись ошибок:

```
command 2> errors.txt
```

Запись обычного вывода и ошибок:

```
command > output.txt 2>&1
```

Скрытие вывода:

```
command > /dev/null 2>&1
```

Конвейеры

Пример подсчёта процессов:


```
ps aux | tail -n +2 | wc -l
```

Поиск строк:

```
cat /etc/passwd | grep "/bin/bash"
```

Более предпочтительная форма:

```
grep "/bin/bash" /etc/passwd
```

Сортировка и удаление повторений:

```
cut -d: -f7 /etc/passwd | sort | uniq
```

Подсчёт разных shell:

```
cut -d: -f7 /etc/passwd | sort | uniq -c
```

Безопасная работа с переменными

Пути и переменные заключайте в двойные кавычки:

```
cd "$directory"  
tar -czf "$archive" "$source"
```

Неправильно:

```
cd $directory
```

Без кавычек путь с пробелами может быть обработан неверно.

Локальные переменные функций

Используйте `local` :

```
create_backup() {  
    local source_dir="$1"  
    local timestamp  
    local archive_path  
}
```

Это уменьшает риск случайного изменения глобальных переменных.

Проверка прав администратора

Некоторые системные журналы могут быть недоступны обычному пользователю.

Проверить запуск от имени `root` можно так:

```
if [ "$EUID" -eq 0 ]; then  
    echo "Скрипт запущен с административными правами"  
else  
    echo "Скрипт запущен обычным пользователем"  
fi
```

Не следует требовать запуск всего скрипта через `sudo` . Основные функции должны работать от обычного пользователя.

Заголовки функций

Для удобного вывода можно создать функцию:

```
print_header() {  
    echo  
    echo "=====  
    echo "$1"  
    echo "=====  
}
```

Пример:

```
print_header "Сетевая информация"
```

Что проверить перед отправкой

Чек-лист

Файлы проекта

- ☐ Основной файл называется `system_helper.sh`.
- ☐ В первой строке указан `#!/bin/bash`.
- ☐ Файл имеет право на выполнение.
- ☐ В проекте отсутствуют временные и ненужные файлы.
- ☐ Скрипт автоматически создаёт директории `logs`, `backups` и `reports`.
- ☐ К проекту добавлен короткий файл `README.txt` или `README.md`.

Структура кода

- ☐ Основные действия оформлены как функции.
- ☐ Для обработки команд используется `case`.
- ☐ Используется хотя бы один цикл.
- ☐ Используются переменные.
- ☐ В функциях используются локальные переменные.
- ☐ Аргументы командной строки обрабатываются через `$1`, `$2` и `$3`.
- ☐ Пути и переменные заключены в двойные кавычки.

- ☐ Код имеет понятные отступы.
- ☐ В коде есть короткие комментарии.

Команды

- ☐ Работает `./system_helper.sh help`.
- ☐ Работает `./system_helper.sh info`.
- ☐ Работает `./system_helper.sh users`.
- ☐ Работает `./system_helper.sh processes`.
- ☐ Работает `./system_helper.sh network`.
- ☐ Работает проверка существующей службы.
- ☐ Корректно обрабатывается неизвестная служба.
- ☐ Работает поиск файлов.
- ☐ Корректно обрабатывается поиск без результатов.
- ☐ Создаётся резервная копия директории.
- ☐ Создаётся системный отчёт.

Обработка ошибок

- ☐ При запуске без команды выводится справка или меню.
- ☐ При неизвестной команде выводится понятная ошибка.
- ☐ При отсутствии аргумента выводится понятная ошибка.
- ☐ Несуществующая директория не приводит к непонятному аварийному завершению.
- ☐ Ошибки выводятся через `>&2`.
- ☐ При ошибке возвращается ненулевой код завершения.
- ☐ При успешном выполнении возвращается код `0`.
- ☐ Ошибки системных команд не засоряют экран лишним выводом.
- ☐ Скрипт не удаляет файлы пользователя.
- ☐ Скрипт не изменяет системные конфигурационные файлы.
- ☐ Скрипт не завершает процессы.
- ☐ Скрипт не останавливает службы.

Логи и результаты

- ☐ Действия записываются в лог.
- ☐ Каждая запись лога содержит дату и время.

- ☐ Резервные копии сохраняются в `backups` .
- ☐ Отчёты сохраняются в `reports` .
- ☐ Имена архивов содержат дату и время.
- ☐ Имена отчётов содержат дату и время.
- ☐ Скрипт сообщает полный путь к созданному файлу.
- ☐ Размер созданного архива или отчёта отображается пользователю.

Проверка синтаксиса

Перед отправкой выполните:

```
bash -n system_helper.sh
```

Если команда ничего не вывела, основные синтаксические ошибки не найдены.

Дополнительно можно проверить скрипт через `shellcheck` , если программа установлена:

```
shellcheck system_helper.sh
```

Советы по улучшению работы

1. Не помещайте весь код в один большой блок

Разделите программу на небольшие функции.

Вместо одного блока на 200 строк используйте отдельные функции:

```
show_system_info  
show_users  
show_processes  
show_network  
create_backup  
create_report
```

Так код проще читать, проверять и исправлять.

2. Используйте понятные имена

Хорошие имена:

```
source_directory  
archive_path  
service_name  
report_file
```

Неудачные имена:

```
a  
b  
x1  
temp2
```

Исключение — короткие переменные внутри очень простых циклов.

3. Не используйте `sudo` внутри каждой команды

Скрипт должен выполнять максимум действий от имени обычного пользователя.

Если конкретной операции не хватает прав, выведите понятное сообщение:

```
Недостаточно прав для чтения полного системного журнала.  
Попробуйте запустить эту команду через sudo.
```

4. Не скрывайте все ошибки

Перенаправление:

```
2>/dev/null
```

используйте только там, где ошибка ожидаема и обрабатывается программой.

В остальных случаях лучше записать ошибку в лог.

5. Проверяйте входные данные

Перед использованием пути проверьте:

- передан ли аргумент;
- существует ли объект;
- является ли он директорией;
- доступен ли он для чтения.

6. Не используйте `eval`

Для данного задания команда `eval` не нужна. Она может привести к выполнению нежелательного кода, если аргументы получены от пользователя.

7. Тестируйте команды отдельно

Перед добавлением команды в скрипт выполните её в терминале.

Например:

```
ps aux --sort=-%cpu | head -n 6
```

После проверки перенесите её в функцию.

8. Проверяйте пути с пробелами

Создайте тестовую директорию:

```
mkdir -p "$HOME/test directory"
touch "$HOME/test directory/file one.txt"
```

Проверьте, что скрипт может создать её резервную копию:

```
./system_helper.sh backup "$HOME/test directory"
```

Если переменные заключены в кавычки, команда должна работать корректно.

9. Добавьте единый стиль вывода

Используйте одинаковые заголовки:

```
=== Информация о системе ===
=== Пользователи ===
=== Процессы ===
=== Сеть ===
```

Успешные операции обозначайте одинаково:

```
[OK] Отчёт создан
```

Ошибки:

```
[ERROR] Директория не существует
```

Предупреждения:

```
[WARNING] Недостаточно прав для чтения журнала
```


10. Добавьте комментарии только к важным местам

Не нужно комментировать очевидные строки.

Необязательный комментарий:

```
# Выводим дату
date
```

Полезный комментарий:

```
# Архивируем только содержимое родительской директории,
# чтобы в архиве не сохранялся полный абсолютный путь.
tar -czf "$archive_path" -C "$(dirname "$source_dir")" "$source_name"
```

Требования к результату

Для сдачи необходимо предоставить:

```
linux-final-project/
├─ system_helper.sh
├─ README.md
├─ backups/
├─ logs/
└─ reports/
```

В `README.md` укажите:

- название проекта;
- назначение скрипта;
- требования для запуска;
- команду выдачи прав на выполнение;
- список доступных команд;

- примеры запуска;
- известные ограничения;
- пример задания для `cron`.

Минимальный запуск проекта:

```
chmod +x system_helper.sh  
./system_helper.sh help
```

Ожидаемый результат

После выполнения задания пользователь должен уметь:

- писать Bash-скрипты;
- разделять скрипт на функции;
- использовать аргументы командной строки;
- применять условия, циклы и `case`;
- проверять файлы и директории;
- работать с процессами и службами;
- получать сетевую информацию;
- использовать перенаправления и конвейеры;
- создавать архивы;
- формировать отчёты;
- записывать действия в лог;
- проверять коды завершения;
- безопасно обрабатывать ошибки.

Главный критерий успешной работы — скрипт должен быть понятным, безопасным и корректно реагировать как на правильные команды, так и на ошибочные входные данные.